Inside Vitess: Engineering Resilience At Scale

Rohit Nayak

Vitess Maintainer, 6yrs

PlanetScale

Vitess

AMS DB

Some At-Scale Users

- Cursor: https://x.com/vmg/status/1961481692817813538
- Slack: https://slack.engineering/scaling-datastores-at-slack-with-vitess/
- Vitess at Github: https://www.youtube.com/watch?v=_Xl24s_0mZs
- Vitess at Uber: https://medium.com/@kansalharshit84/why-uber-moved-from-postgres-to-mysql-a-controversial-shift-in-the-world-of-databases-b71016be3f62
- Vinted and Vitess: https://vinted.engineering/2023/04/27/vinted-vitess-voyage-chapter-1-autumn-is-coming/

Vitess

AMS DB

Idea of Scale

- Slack: millions of messages, ~3 million QPS
- Multiple 256 / 512-way sharded databases
- Square: billions of payment transactions per month
- Github: all pulls/issues, 5.5 million QPS, 1200+ MySQL, 500TB
- Planetscale: peak QPS: 10s of millions, thousands of clusters
- Finance DB > 10PB
- Migrations of 1B rows/hour

Vitess

AMS DB

- On Resilience
- What is Vitess?
- Fault Vectors
- Resilience in Vitess
- Summary

Vitess

AMS DB

Availability: how many 9s are enough?

Availability	Downtime / Year	Downtime / Month	Downtime / Week	Downtime / Day
99.999%	5.256 Minutes	0.438 Minutes	0.101 Minutes	0.014 Minutes
99.995%	26.28 Minutes	2.19 Minutes	0.505 Minutes	0.072 Minutes
99.990%	52.56 Minutes	4.38 Minutes	1.011 Minutes	0.144 Minutes
99.950%	4.38 Hours	21.9 Minutes	5.054 Minutes	0.72 Minutes
99.900%	8.76 Hours	43.8 Minutes	10.108 Minutes	1.44 Minutes
99.500%	43.8 Hours	3.65 Hours	50.538 Minutes	7.2 Minutes
99.250%	65.7 Hours	5.475 Hours	75.808 Minutes	10.8 Minutes
99.000%	87.6 Hours	7.3 Hours	101.077 Minutes	14.4 Minutes

Source: https://aws.amazon.com/blogs/publicsector/achieving-five-nines-cloud-justice-public-safety/

Measures

- Downtime
- Latency
- Throughput (QPS)

Implications

- Cost from loss of service
- Critical services become unavailable
- Loss of data

Ground Realities

- Lack of personnel
- Need to keep costs low
- Time to market

Vitess

AMS DB

The Oct 19th-20th AWS Outage:

approx. per hour loss

Amazon:	$72,831,050
Snapchat:	$611,986
Zoom:	$532,580
Roblox:	$411,187
Fortnite:	$399,543
Canva:	$342,466
Slack:	$194,064
Reddit:	$148,402

source: Economic Times

Vitess

AMS DB

- Why Resilience?
- What is Vitess?
- Fault Vectors
- Resilience in Vitess
- Summary

Vitess

AMS DB

Vitess is a massively scalable

highly available

distributed database system

built around MySQL

Vitess

What is Vitess?

- Cloud Native Database
- Massively Scalable
- Highly Available
- MySQL 8.0/8.4 Compatible

Logical Database

- Many physical databases
- Query Routing
- gRPC Clients MySQL protocol
- Single Connection

Works With

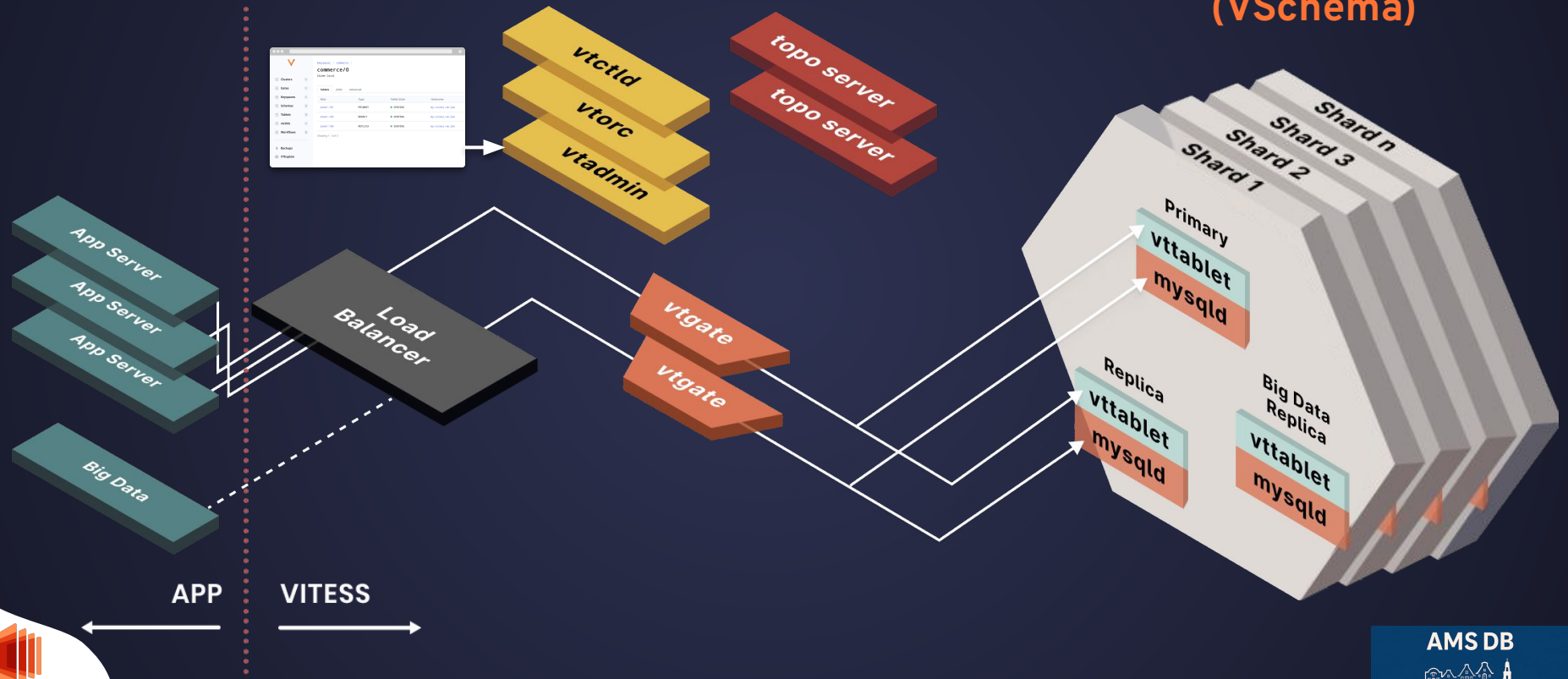
- Database Frameworks
- ORMs
- Legacy Code
- Third-Party Applications

slack
New Relic
Square
Flipkart
HubSpot
peak
JD.COM 京东
Etsy
nozzle
weave
GitHub
Uber
Quiz of Kings
stitchlabs
PlanetScale

Vitess

AMS DB

Architecture



Vitess

AMS DB

- Why Resilience?
- What is Vitess?
- Fault Vectors
- Resilience in Vitess
- Summary

Vitess

AMS DB

Hardware And Infrastructure

disk_failure
vm_crash_or_reboot
disk_full
hardware_fault
cpu_throttling_or_overheating
memory_leak

Network And Connectivity

network_partition
dns_resolution_failure
dropped_packets
connection_leak_or_reset
timeout_misconfiguration
duplicate_packets
high_latency_or_jitter

Software And Runtime

application_bug
leader_election_failure
panic_or_crash
long_gc_pause
process_freeze_or_hang
serialization_issue
cpu_overuse
data_inconsistency
deadlock_condition
out_of_memory_error

Database Behavior And Logic

replication_lag
cluster_split_brain
slow_query_node
index_corruption
stale_or_inconsistent_data
clock_sync_issue
transaction_conflict
write_amplification
checkpoint_stall

Load And Resource Pressure

unexpected_traffic_spike
load_spike
high_cpu_usage
resource_exhaustion
connection_pool_exhaustion
slow_node_under_load
request_queue_backlog
memory_exhaustion

Human And Configuration Errors

misconfiguration
manual_mistake
missed_security_patch
invalid_or_expired_certificate
untested_backup
missing_monitoring_or_alerts
bad_timeout_settings
accidental_deletion_of_resources

Vitess

AMS DB

- Why Resilience?
- What is Vitess?
- Fault Vectors
- Resilience in Vitess
- Summary

Vitess

AMS DB

Resilience Principles

- Isolation (Bulkheads) `stateless/shared nothing, no cascading failures`
- Redundancy `multiple copies of data, over-provisioning`
- Graceful Degradation `higher latency (vs failure)`
- Circuit Breakers `throttling, drop new connections`
- Automated Recovery `proactive(auto-scaling), reactive(self-healing)`
- Observability `logging, metrics, alerts`
- Testing `chaos tests, fuzzing, e2e/unit tests, phased (blue/green)releases`

Vitess

AMS DB

Isolation

- Cells: Multiple Availability Zones `us-east-1, eu-west-2, ap-south-1`
- Multi-Cloud `AWS, GCP, Azure`
- Replicas in different AZs
- VTGates are stateless
- topo not in query-serving path
- global & per-cell local topo
- Sharded Database `shared-nothing, per-shard metadata`

Vitess

AMS DB

Redundancy

- MySQL Replication
- Semi-sync for durable writes
- Sharding `limit data unavailability`
- Distributed topo cluster `3/5/7 nodes, raft consensus`

Vitess

AMS DB

Graceful Degradation

- VTTablet: Query Consolidation
- VTTablet: Connection Pooling
- Cell Isolation: `local cell reads`
- VTTablet: Backpressure
  - Migration Throttling
  - Query Throttling
  - Timeouts `load shedding`
- VTGate: Buffering `reparents, migration cutovers`
- VTGate: Replica Reads `ignore lagged replicas`

Vitess

AMS DB

Automated Recovery (Self Healing)

- Health Checks `replica lag, primary reparent`
- vtOrc `observes cluster health, emergency reparent`
- Backup/Restore `pitr, incremental backups, smaller shard sizes`
- Reversible Workflows `schema changes, import, reshard`
- VTOP: replacement over repair `bad mysql, k8s node, vtgate oom`

Vitess

AMS DB

Operations

- Telemetry `prometheus metrics`
- Paging alerts `replica lag, memory pressure, qps, latency`
- Regular Updates `dependabot, golang versions`

Vitess

AMS DB

Testing/Benchmarking

- arewefastyet: per-PR opt-in benchmarking
- Fuzzing tests
- Per-commit CI
- Comprehensive e2e tests
- Good unit test coverage

Vitess

AMS DB

- Why Resilience?
- What is Vitess?
- Fault Vectors
- Resilience in Vitess
- Summary

Vitess

AMS DB

Resilient Architecture

- MySQL as storage engine: inherits proven durability
- Topology Service (etcd) as source of truth. Global + per-cell
- Sharded MySQL: limit single-node failure impact
- Multiple Cells => Availability Zones
- Stateless query serving (VTGate) layer
  - Load balanced
  - Topo not in hot query path
- Tablet Layer: primary/replica. Supervises MySQL server
  - Health checks, Reparenting
  - Query throttling, Result caching
  - Reject bad queries
- Vitess Orchestrator (VTOrc): ~zero downtime failovers
- VReplication Control Plane, for all schema/data migration

Vitess

AMS DB

Resilient Design

- Failover Buffering at VTGate: during cluster events like reparenting
- Health-checking & Discovery: vtgate filters out unhealthy tablets
- Tablet Throttler: backpressure on VReplication on high replication lag
- Cross-shard Transactions (2PC): for consistency
- Cell Isolation: local-cell reads on network partition
- Operational guardrails: Dry-runs, Reversible migrations, Timeouts
- Controlled failovers: for rolling upgrades
- Backups: incremental, point-in-time recovery

Vitess

AMS DB

Resilient Mechanisms

- Flags & knobs: tailor for specific environments and use-cases
  `read-after-write, stale replicas, buffer size, semi-sync`
- Per-shard metadata store: persistent distributed internal state
- Health probes: self-access lag using heartbeats
- Monitoring hooks: per component /debug/vars, Prometheus metrics

Vitess

AMS DB

In Conclusion

Vitess's resilience comes from a combination of **deliberate design decisions**:

- leveraging using tried-and-tested MySQL replication for redundancy
- using an intelligent routing layer for transparent failover.
- partitioning the system into shards and cells
- automating failover/recovery tasks.

Standard fault-tolerance patterns

- redundant components
- failover automation
- graceful degradation
- isolation
- eventual consistency for performance

Vitess

AMS DB

Thanks!

Questions?

Vitess

AMS DB